EDUCAÇÃO PROFISSIONAL PAULISTA • TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Avançando na integração com serviços externos

Uma trilha de estudo completa para compreender como uma interface front-end conversa com serviços externos, solicita dados, autentica usuários e informa o que está acontecendo na tela.

API → REST → GraphQL → OAuth → JWT → Feedback visual

ETAPA INICIAL

Identificação do estudante

Esses dados serão usados no relatório final e no PDF de entrega.

Nome completo

Arthur Saraiva De Franca

Turma

3b

Nº da chamada (opcional)

3

Data

05/10/2026

Salvar identificação

MAPA DA APRENDIZAGEM

Trilha da Semana 22

Você vai partir da comunicação com APIs, comparar REST e GraphQL e depois estudar autenticação com OAuth e JWT.

✓ 1. Identificação Concluído	2. Aula 1 — REST Pendente	3. Aula 2 — GraphQL Pendente
4. Aula 3 — OAuth/JWT Pendente	✓ 5. Atividade prática Concluído	6. Quiz final Pendente

AULA 1

Integração com APIs RESTful

A aula apresenta REST como um estilo arquitetural para construir APIs de forma padronizada, com requisições independentes, recursos identificados por URLs, uso de métodos HTTP e respostas frequentemente em JSON.

1

Antes de REST: o que é uma API?

API é uma forma organizada de permitir que dois sistemas troquem informações ou utilizem funcionalidades um do outro.

Explicação técnica

O front-end envia uma **requisição** para um serviço. Esse serviço processa o pedido e devolve uma **resposta**. A API define quais caminhos, dados e regras podem ser utilizados nessa comunicação.

💡 Pense assim: o garçom

Você não entra na cozinha para preparar seu prato. Você faz o pedido ao garçom, ele leva a solicitação à cozinha e depois traz a resposta. A API faz o papel desse intermediário entre sistemas.

🌎 Onde isso aparece na vida real? Aplicativos de clima, mapas, pagamentos, envio de mensagens, login social e consulta de produtos usam integrações com serviços externos.

🖥️

Cliente

É quem solicita algo. Em nosso contexto, normalmente o navegador ou o front-end.

Analogia: a pessoa que faz o pedido.

🏢

Servidor

Recebe a requisição, processa e devolve uma resposta.

Analogia: a cozinha que prepara o pedido.

🚪

Endpoint

É um endereço específico disponibilizado pela API, como `/produtos`.

Analogia: uma porta para um setor específico.

🔗

URL

Identifica o recurso que queremos acessar, por exemplo `/users/1`.

Analogia: o endereço para encontrar o destino.

↔

Stateless

Cada requisição é independente; ela precisa trazer o necessário para ser entendida.

Analogia: cada pedido vem com suas próprias instruções.

{ }

JSON

Formato textual muito usado para organizar e transportar dados.

Analogia: uma ficha com campos e valores.

ENTENDENDO OS DADOS

Como ler um JSON?

JSON organiza informações em pares de **chave** e **valor**. Veja um exemplo simples de produto.

`"nome"` → chave que identifica a informação.

`"Mouse"` → valor associado à chave.

`79.90` → valor numérico.

```
{
  "nome": "Mouse",
  "preco": 79.90,
  "disponivel": true
}
```

`true` → valor lógico.

2

Métodos HTTP: o que você quer fazer?

O método ajuda a indicar a intenção da requisição.

> 💡 **Pense em uma loja:**
>
> consultar, cadastrar, atualizar e remover são ações diferentes. Os métodos HTTP ajudam a expressar essas intenções.

GET	POST	PUT	DELETE
Consultar / ler	Criar	Atualizar	Remover

DELETE — remoção

Usado para remover um recurso.

```
DELETE /users/1
```

PASSO A PASSO

O que acontece em uma integração?

1 Usuário solicita algo na interface

2 Front-end cria a requisição

3 API recebe o pedido

4 Servidor processa

5 Resposta retorna em dados

6 Front-end atualiza a tela

📖 Por que a documentação de uma API é importante?

A documentação orienta como realizar a integração: quais recursos existem, quais caminhos utilizar, qual método corresponde a cada ação, quais dados enviar e que tipo de resposta esperar.

> 💡 **Pense assim: manual de instruções**
>
> Sem o manual você pode até tentar usar um equipamento, mas perde tempo experimentando. A documentação reduz tentativas e ajuda a seguir as regras do serviço.

Endpoints disponíveis	**Métodos aceitos**	**Dados de entrada**
Formato da resposta	**Autenticação**	**Possíveis erros**

▶ 🤔 **Pare e pense: por que usar uma API pronta?**

SIMULADOR

Requisição REST

Clique para visualizar uma requisição de leitura e uma resposta JSON simulada.

Executar GET

```
GET /api/produtos
Enviando requisição...
200 OK
{
  "id": 1,
  "nome": "Produto",
  "preco": 49.9
}
```

Quiz — Aula 1

0/5

1. Uma tela precisa buscar a lista de produtos sem criar ou alterar nada. Qual método é o mais adequado?

- ○ POST
- ◉ GET
- ○ PUT
- ○ DELETE

---

2. Um estudante diz: "Depois da primeira requisição, o servidor REST precisa lembrar tudo sobre o cliente para entender a próxima". Qual conceito mostra o problema dessa afirmação?

- ○ JSON
- ◉ Stateless
- ○ Endpoint
- ○ PUT

---

3. A URL /users/1 representa, no exemplo da aula:

- ◉ Um recurso específico identificado por URL
- ○ Um arquivo de CSS
- ○ Um método HTTP
- ○ Uma chave de API

4. Você precisa integrar uma API desconhecida e quer descobrir endpoints, métodos e formato de uso. O que deve consultar primeiro?

- (●) A documentação da API
- ( ) O arquivo CSS
- ( ) Um editor de imagens
- ( ) Apenas o console do navegador

5. Por que usar uma API pronta pode economizar trabalho?

- (●) Porque permite adicionar funcionalidades sem desenvolver tudo do zero
- ( ) Porque elimina todo o código do sistema
- ( ) Porque garante que nenhum erro aconteça
- ( ) Porque substitui qualquer back-end

AULA 2

Integração com GraphQL

O material avança para GraphQL, destacando consultas eficientes, definição de esquema, tipos, consultas e relacionamentos, além da possibilidade de recuperar apenas os dados necessários.

1

Qual problema o GraphQL ajuda a resolver?

Em algumas telas, o front-end precisa de um conjunto específico de dados. GraphQL permite descrever na própria consulta quais campos devem retornar.

Explicação técnica

Em GraphQL, o cliente envia uma **query** descrevendo os campos desejados. A consulta é validada com base em um **schema**, que define os tipos de dados e as operações disponíveis.

> 💡 **Pense assim: monte seu pedido**
>
> REST pode lembrar um combo já definido. GraphQL pode lembrar um pedido em que você seleciona exatamente os itens que deseja receber.

REST

Resposta definida pelo endpoint

```
GET /usuarios/1
```

O endpoint possui uma estrutura de resposta previamente definida pelo serviço.

GraphQL

Campos definidos na consulta

```
query {
  usuario {
    nome
    email
  }
}
```

O cliente declara quais campos deseja obter naquela consulta.

Importante: isso não significa que GraphQL seja sempre “melhor”. REST e GraphQL são abordagens diferentes e podem ser adequadas a contextos diferentes.

🔎

Query

É a consulta enviada para solicitar dados.

Analogia: a lista do que você deseja pedir.

📐

Schema

Descreve os tipos e operações disponíveis.

Analogia: a planta da casa mostrando o que existe.

🧩

Type

Define a estrutura de determinado conjunto de dados.

Analogia: um molde que diz quais características um objeto possui.

🔗

Relacionamento

Mostra como informações diferentes se conectam.

Exemplo: evento → palestrante → local.

📐 Schema como planta de uma casa

Uma planta mostra quais cômodos existem e como se conectam. No GraphQL, o schema descreve quais tipos existem, quais campos cada tipo possui e quais consultas podem ser realizadas.

```
type Evento {
  titulo: String
  palestrante: Palestrante
  local: Local
}
```

EXPLORADOR VISUAL

Monte uma consulta

Selecione apenas os dados que você precisa:

- [x] **nome**
- [x] **email**
- [x] **telefone**

Query

```
query {
  usuario {
    nome
    email
    telefone
```

```
  }
}
```

Resposta simulada

```
{
  "data": {
    "usuario": {
      "nome": "Marina",
      "email": "marina@email.com",
      "telefone": "(11) 99999-0000"
    }
  }
}
```

▼ 🤔 **Pare e pense: você precisa de 30 campos, mas usa apenas 2?**

Uma consulta personalizada pode ser útil quando a interface precisa de um subconjunto específico de informações. A Aula 2 destaca justamente a recuperação apenas dos dados necessários.

🌎 **Onde isso aparece na vida real?**	Interfaces com várias telas podem precisar de combinações diferentes de dados. GraphQL permite que cada consulta expresse os campos necessários para aquela situação.

Quiz — Aula 2

0/5

1. Uma interface precisa apenas de nome e email, mas o serviço possui muitos outros campos. Qual ideia da Aula 2 combina melhor com esse caso?

- ◉ Solicitar apenas os dados necessários com GraphQL
- ○ Sempre baixar todos os dados

- ○ Eliminar o schema
- ○ Trocar JavaScript por CSS

---

2. No GraphQL, qual elemento descreve tipos, consultas e relacionamentos disponíveis?

- ◉ Schema
- ○ Spinner
- ○ JWT
- ○ Endpoint REST

---

3. Qual alternativa descreve melhor uma query GraphQL?

- ◉ Uma consulta que declara os dados desejados
- ○ Um arquivo de estilo
- ○ Uma senha do servidor
- ○ Um método de exclusão de banco

---

4. Qual ambiente é citado na Aula 2 para uma API simples com GraphQL?

- ◉ Node.js e Express
- ○ Photoshop e Illustrator
- ○ Excel e Access

- ○ Somente HTML

---

5. REST e GraphQL devem ser entendidos como:

- ◉ Abordagens diferentes de integração e consulta
- ○ Tecnologias em que uma sempre substitui a outra
- ○ Ferramentas exclusivamente de design
- ○ Formatos de imagem

AULA 3

Autenticação com OAuth e JWT

A aula trabalha autenticação segura, login com conta de terceiros como Google, uso de JWT e feedback visual para tornar o processo de login mais claro para o usuário.

Autenticação

Quem é você?

É o processo de verificar a identidade de quem está tentando entrar.

> 💡 **Escola:**
> mostrar sua carteirinha para confirmar sua identidade.

Autorização

O que você pode fazer?

Depois de saber quem é o usuário, o sistema pode controlar quais recursos ele pode acessar.

> 💡 **Escola:**
> saber quais salas e sistemas aquele usuário tem permissão para acessar.

1

OAuth: autenticação com um serviço confiável

No material, Google aparece como exemplo de conta de terceiros utilizada para login.

Explicação simples

Em vez de criar uma nova conta e senha em cada aplicação, o usuário pode utilizar um provedor conhecido para realizar o processo de autenticação.

A aplicação não precisa "receber a senha do Google" para usar o fluxo OAuth. O processo ocorre por redirecionamentos e tokens controlados pelo provedor.

> 💡 **Pense assim: uma credencial confiável**
>
> É como apresentar uma credencial emitida por uma instituição reconhecida. A aplicação confia no processo realizado pelo provedor e recebe uma resposta de autenticação.

CONCEITO DA ATIVIDADE

`client_id`: quem está pedindo o login?

Na análise proposta no roteiro, o aluno precisa identificar a finalidade do `client_id`. Ele representa a aplicação cadastrada no provedor OAuth e ajuda o serviço a reconhecer quem está solicitando o fluxo de autenticação.

> 💡 **Pense assim: o crachá da aplicação**
>
> Quando a aplicação chega ao "balcão" do Google para iniciar o login, ela precisa se identificar. O `client_id` funciona como esse identificador.

Usuário → **Front-End** → **Google OAuth** → **Back-End** → **JWT** → **Sessão autenticada**

SIMULADOR DIDÁTICO

1. Redirecionando para o provedor de login...
2. Aplicação exibe o estado autenticado.

O que acontece ao clicar em “Entrar com Google”?

O simulador não faz autenticação real. Ele mostra a sequência de etapas para você visualizar o fluxo.

G Entrar com Google

3. Token do provedor é recebido...
4. Usuário conclui a autenticação...
5. Token é enviado ao back-end...
6. Token do provedor é recebido...
7. Back-end valida e retorna um JWT...
8. Token é enviado ao back-end...
9. Aplicação exibe o estado autenticado.
10. Back-end valida e retorna um JWT...
11. Aplicação exibe o estado autenticado.

2

JWT: como a aplicação reconhece uma sessão autenticada?

O roteiro pede que o estudante explique o propósito do JWT gerado pelo back-end depois da validação do token do Google.

JWT é um formato de token compacto. Em um fluxo de autenticação, ele pode ser usado pelo back-end para representar informações da sessão e permitir que requisições posteriores sejam reconhecidas como autenticadas.

Importante: token não deve ser tratado como se fosse uma senha comum nem exposto sem necessidade.

> 💡 **Pense assim: pulseira de um evento**
>
> Depois que sua entrada é validada, você recebe uma pulseira. Em áreas autorizadas, ela serve como sinal de que você já passou pela verificação.

VISUALIZAÇÃO CONCEITUAL

JWT em três partes

`HEADER . PAYLOAD . SIGNATURE`

Header
Metadados sobre o token e seu formato.

Payload
Informações/claims transportadas no token.

Signature
Parte utilizada para verificar integridade/autenticidade.

Demonstração didática. Não cole tokens reais nesta página.

🎛️ Por que loading, sucesso e erro importam no front-end?

O material destaca que o usuário precisa receber feedback enquanto aguarda o Google ou o servidor. Sem esse retorno visual, ele pode pensar que o sistema travou ou clicar várias vezes.

> 💡 **Pense assim: "seu pedido está sendo preparado"**
>
> Um spinner ou mensagem de carregamento informa que existe uma ação em andamento.

Normal
Pronto para iniciar.

Loading
Aguardando resposta.

Sucesso
Operação concluída.

Erro
Algo falhou e o usuário precisa ser orientado.

EXPERIÊNCIA DO USUÁRIO

Teste os estados da autenticação

Normal | Carregando | Sucesso | Erro

✓ Autenticado

Login realizado com sucesso.

❌ **Mensagem pouco amigável** ✅ **Mensagem orientativa**

```
Error 500 OAuth callback exception
```

```
Não foi possível realizar o login.
Verifique sua conexão e tente novamente.
```

Quiz — Aula 3

0/5

1. Ao clicar em "Entrar com Google", qual conceito da Aula 3 está sendo aplicado?

- (●) Autenticação com OAuth
- ( ) Apenas estilização CSS
- ( ) Consulta GraphQL
- ( ) Método DELETE

---

2. Na atividade, o client_id deve ser analisado porque ele ajuda a:

- (●) Identificar a aplicação no provedor OAuth
- ( ) Guardar a senha do usuário
- ( ) Definir o CSS da página
- ( ) Substituir o JWT

---

3. Depois que o back-end valida o token do Google, o roteiro pede que o aluno explique o propósito de qual elemento?

- (●) JWT
- ( ) GET

- ○ Schema
- ○ Spinner

---

4. Se o login está demorando e a tela não mostra nada, qual melhoria está alinhada ao material?

- ◉ Exibir um indicador de carregamento
- ○ Ocultar ainda mais informações
- ○ Recarregar a página continuamente
- ○ Remover o botão de login

---

5. Se a autenticação falhar, qual comportamento melhora a experiência do usuário?

- ◉ Mostrar uma mensagem de erro clara e permitir nova tentativa
- ○ Não informar nada
- ○ Exibir apenas uma exceção técnica
- ○ Fechar o navegador

PRÁTICA GUIADA

Laboratório Front-End

Agora os conceitos aparecem em pequenos trechos de código. A ideia é entender o papel de cada linha antes de pensar em programas maiores.

REST

Consumindo um recurso

```
fetch('/api/produtos')
  .then(resposta => resposta.json())
  .then(dados => console.log(dados));
```

`fetch()` inicia a requisição.

`resposta.json()` interpreta o corpo como JSON.

O segundo `.then()` trabalha com os dados recebidos.

> 💡 **Analogia:**
> você faz um pedido e combina o que fará quando a entrega chegar.

UX

Estado de carregamento

```
setLoading(true);
// aguarda a operação
setLoading(false);
```

`true` indica que a operação começou.

`false` indica que ela terminou.

> 💡 **Analogia:**
> acender e apagar a placa “atendimento em andamento”.

Erro

Feedback para o usuário

```
setError(
  'Erro na autenticação. Tente novamente.'
);
```

O estado guarda uma mensagem que pode ser exibida na interface.

O objetivo é orientar o usuário em vez de deixá-lo sem resposta.

Token

Exemplo de armazenamento local

```
localStorage.setItem('token', jwt);
```

Este é apenas um exemplo didático ligado à pergunta do roteiro sobre armazenamento e uso do JWT no front-end.

Em aplicações reais, decisões de armazenamento exigem análise de segurança do projeto.

ATIVIDADE PRÁTICA

Analisando a integração de autenticação em uma aplicação front-end

O roteiro propõe analisar uma solução com Google OAuth e JWT e depois melhorar o tratamento de erros e o feedback visual.

Antes de responder

Relembre: o `client_id` identifica a aplicação cadastrada no provedor OAuth. Sua resposta deve explicar a finalidade e localizar onde ele aparece no fluxo/código estudado.

1.

Qual a finalidade do

`client_id`

na implementação do OAuth e onde ele é utilizado dentro do código da aplicação?

O client_id identifica a aplicação no Google. Ele é usado na URL de login do OAuth.

Antes de responder

A função `handleGoogleLogin` está relacionada ao início do redirecionamento do usuário para o provedor de autenticação. Observe no material/código quais parâmetros formam a URL.

2.

Explique como a função

`handleGoogleLogin`

funciona para redirecionar o usuário para o login do Google. Qual URL é gerada para a autenticação?

A handleGoogleLogin cria a URL de autenticação do Google e redireciona o usuário para fazer login.

📚 **Antes de responder**

Pense na troca entre front-end e back-end: um token recebido no fluxo precisa ser enviado para validação, e o servidor devolve uma resposta própria para a aplicação.

3.

Descreva o que ocorre na função

`handleTokenExchange`

. Como o token do Google é enviado para o back-end, e o que o back-end retorna?

A handleTokenExchange envia o token do Google para o back-end. O back-end valida o token e retorna um JWT.

📚 **Antes de responder**

Relembre o papel do JWT depois que o back-end valida a autenticação: ele ajuda a aplicação a reconhecer requisições/autorização da sessão.

4.

Qual é o propósito do JWT gerado pelo back-end após a validação do token do Google? Como ele é armazenado e utilizado no front-end?

O JWT identifica a sessão autenticada do usuário. No front-end, ele pode ser armazenado no localStorage e usado nas requisições autenticadas.

PROPOSTA DE MELHORIA

Tratamento de erros e feedback visual

O roteiro orienta usar estado de erro, mensagem visível em caso de falha e indicador de carregamento enquanto o usuário aguarda o Google ou o servidor.

- [ ] **Mensagem de erro**
- [x] **Indicador de carregamento**
- [ ] **Spinner**
- [ ] **Botão desabilitado durante a operação**

Descreva a melhoria que você implementou

Adicionei um indicador de carregamento.

Cole aqui o trecho principal do código

```
import React, { useState } from 'react';

function App() {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [sucesso, setSucesso] = useState(false);

  async function handleGoogleLogin() {
    setLoading(true);
    setError('');
    setSucesso(false);

    try {
      // Simula a autenticação
      await new Promise(resolve => setTimeout(resolve, 2000));
```

Copiar código Limpar código

EVIDÊNCIAS

Telas e melhorias implementadas

Selecione imagens para pré-visualizar e incluir no PDF enquanto esta página permanecer aberta.

Escolher arquivos 3 arquivos

Captura de tela 2026-10-05 081644.png

Captura de tela 2026-10-05 081718.png

Captura de tela 2026-10-05 081729.png

RELATÓRIO

Registro final

O roteiro solicita um relatório com as respostas, o código e as telas das melhorias implementadas.

Conclusão / síntese do relatório

Aprendi como o front-end se comunica com APIs usando REST e GraphQL, além de entender OAuth, JWT e a importância de mostrar mensagens de carregamento e erro para o usuário.

REVISÃO GERAL

Teste seus conhecimentos

Agora misturamos conceitos e situações para verificar se você entende quando e por que cada tecnologia aparece.

Mapa mental da semana

Front-End → **Serviços externos** → **REST / GraphQL** → **Autenticação** → **OAuth** → **JWT** → **Feedback visual**

API
permite que sistemas conversem.

REST
organiza recursos e ações por HTTP.

GraphQL
permite solicitar campos específicos.

OAuth

coordena autorização/autenticação com outro serviço.

JWT
é um formato compacto de token.

Feedback
informa loading, sucesso e erro.

Quiz geral — Semana 22

0/10

1. Uma tela quer apenas consultar dados de produtos. Qual combinação está correta?

- (●) GET + endpoint de produtos
- ( ) POST + DELETE
- ( ) JWT + CSS
- ( ) GraphQL + imagem PNG

2. Qual conceito explica que cada requisição REST deve ser compreensível de forma independente?

- (●) Stateless
- ( ) Payload
- ( ) Spinner
- ( ) Schema

3. Qual é o papel mais direto da documentação de uma API?

- ◉ Explicar como realizar a integração corretamente
- ○ Eliminar a necessidade de programar
- ○ Gerar automaticamente toda a interface
- ○ Armazenar tokens

---

4. No GraphQL, o front-end pode declarar quais campos deseja receber por meio de uma:

- ◉ Query
- ○ Folha de estilo
- ○ Imagem
- ○ Senha

---

5. O schema GraphQL é importante porque:

- ◉ Define tipos, consultas e relacionamentos
- ○ Exibe o spinner
- ○ É uma chave de API
- ○ Substitui todo servidor

---

6. Qual situação combina com OAuth no conteúdo estudado?

- ◉ Login com uma conta de terceiro, como Google
- ○ Escolher cor de botão

- ○ Formatar JSON
- ○ Criar um endpoint DELETE

---

7. No fluxo estudado, o JWT aparece depois de:

- ◉ O back-end validar o token do Google
- ○ A página carregar o CSS
- ○ Uma query GraphQL falhar
- ○ O usuário fechar a página

---

8. Um spinner durante autenticação serve principalmente para:

- ◉ Informar que a operação está em andamento
- ○ Aumentar a segurança criptográfica
- ○ Substituir o token
- ○ Criar um novo usuário

---

9. Qual mensagem é mais adequada para um usuário após uma falha de login?

- ◉ Não foi possível realizar o login. Tente novamente.
- ○ OAuth callback exception 500 stack trace
- ○ Nenhuma mensagem
- ○ Senha do servidor inválida: linha 83

10. Ao final da atividade prática, o roteiro solicita:

- (●) Relatório com respostas, código e telas das melhorias
- ( ) Somente um print do quiz
- ( ) Apenas uma frase
- ( ) Excluir o projeto

ENTREGA

Revisar e gerar PDF

Confira suas respostas antes de criar o arquivo para anexar na plataforma.

⚠ **Existem etapas pendentes:**

- Quiz final

Identificação

Nome: Arthur Saraiva De Franca
Turma: 3b
Nº: 3
Data: 2026-10-05

Resultados

REST: —/5
GraphQL: —/5
OAuth/JWT: —/5
Quiz geral: —/10

Atividade

Q1: O client_id identifica a aplicação no Google. Ele é usado na URL de login do OAuth.

Melhoria

Adicionei um indicador de carregamento.

Q2: A handleGoogleLogin cria a URL de autenticação do Google e redireciona o usuário para fazer login.

Q3: A handleTokenExchange envia o token do Google para o back-end. O back-end valida o token e retorna um JWT.

Q4: O JWT identifica a sessão autenticada do usuário. No front-end, ele pode ser armazenado no localStorage e usado nas requisições autenticadas.

```
import React, { useState } from 'react';

function App() {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [sucesso, setSucesso] = useState(false);

  async function handleGoogleLogin() {
    setLoading(true);
    setError('');
    setSucesso(false);

    try {
      // Simula a autenticação
      await new Promise(resolve => setTimeout(resolve, 2000));

      setSucesso(true);
    } catch (erro) {
      setError('Não foi possível realizar o login. Tente novamente.');
    } finally {
      setLoading(false);
    }
  }

  return (
    <div>
      <h1>Login</h1>

      <button
        onClick={handleGoogleLogin}
        disabled={loading}
      >
        {loading ? 'Autenticando...' : 'Entrar com Google'}
      </button>

      {loading && <p>⏳ Aguarde, realizando login...</p>}

      {sucesso && (
        <p>✅ Login realizado com
```

```
sucesso!</p>
      )}

      {error && (
        <p>❌ {error}</p>
      )}
    </div>
  );
}

export default App;
```

Conclusão do relatório

Aprendi como o front-end se comunica com APIs usando REST e GraphQL, além de entender OAuth, JWT e a importância de mostrar mensagens de carregamento e erro para o usuário.

Evidências nesta sessão

3 imagem(ns) selecionada(s).

As respostas ficam armazenadas localmente no navegador. O site não envia suas respostas para um servidor.

Programação Front-End • Semana 22

Integração com serviços externos e esteiras de entrega